

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



LẬP TRÌNH NHÂN LINUX PHẦN 2. TỔNG QUAN

Ngành: Công nghệ thông tin

Chuyên ngành: Kỹ thuật phần
mềm nhúng và di động

Mã số: 52.48.02.01

Nội dung

- Tổng quan về signal
- Xử lý khi nhận signal
- Signal table
- Một số signal cơ bản
- Send signal
- Thực hành

Tổng quan về signal

- Là software interrupt
- Là phương pháp giao tiếp phổ biến nhất giữa các app
- Không thể lường trước được lúc nào thì nhận được signal
- Có 32 giá trị khác nhau, mỗi giá trị được gán 1 tên riêng biệt
 - <http://man7.org/linux/man-pages/man7/signal.7.html>

3

Các trường hợp xảy ra signal

- User sử dụng kill command
- Process gửi signal
- Chương trình gặp lỗi
- User ấn 1 số tổ hợp phím đặc biệt

Phân loại signal

- Signal có thể ignore
- Signal không thể ignore
- Signal có thể chủ động điều khiển được
- Signal không thể chủ động điều khiển được

Signal table

- Mỗi một process sẽ có 1 bảng signal table
 - ❑ Trong đó mỗi hàng của bảng sẽ chứa số hiệu signal và con trỏ hàm trỏ đến handler function của signal đó. Mỗi khi có 1 signal gửi đến, OS sẽ gọi ra signal handler tương ứng.
 - ❑ Con trỏ trỏ đến signal table của process có thể xem tại `struct task_struct->pending` (`include/linux/sched.h`)
- Hệ thống tự tạo ra bảng signal table khi khởi tạo process
- Khi mới được khởi tạo, tất cả các signal handler sẽ là default handler

Đăng ký signal handler

- Function tương ứng:
 - `signal.h`
 - `sighandler_t signal(int signum, sighandler_t handler)`
 - `void sig_handler(int signo)`

Send signal

- `int kill(pid_t pid, int signo)`
 - `pid > 0`, `pid == 0`, `pid < 0`, `pid == -1`
- `int raise(int signo)`
- kill command
- Permission to send a signal
 - Khi gửi signal đặc biệt ví dụ như SIGKILL đến process thuộc user khác thì chương trình cần quyền root để làm việc đó

Thực hành

- Viết 1 chương trình C để in chữ hello world mỗi khi user nhấn phím Ctrl + C
- Viết 1 chương trình C sleep vô thời hạn, sử dụng 1 chương trình C khác gửi signal để wakeup chương trình C đầu tiên lên.

Hoạt động

- Khi 1 signal handler đang chạy, nếu 1 signal khác có độ ưu tiên cao hơn gửi đến, OS sẽ dừng handler hiện tại và thực hiện hàm handler mới.
 - Tuy nhiên OS sẽ không save stack của handler hiện tại lại.
 - Do đó 1 số function sẽ gây lỗi hệ thống nếu bị ngắt giữa chừng và không được hoàn thành. Ví dụ như hàm malloc.
- Linux đã liệt kê 1 số function là an toàn cho signal handler gọi. Các function này không gây lỗi nếu bị stop giữa chừng. Chúng được gọi là Reentrant Functions.

Một số signal cơ bản

Tên signal	Số hiệu signal	Miêu tả
SIGHUP	1	Trì hoãn việc kiểm tra trên quản lý terminal hoặc sự dừng của quản lý tiến trình.
SIGINT	2	Được thông báo nếu người sử dụng gửi một tín hiệu ngắt (Ctrl+C).
SIGQUIT	3	Được thông báo nếu người sử dụng gửi một tín hiệu bỏ (Ctrl+D).
SIGFPE	8	Được thông báo nếu một hoạt động thuộc về toán không hợp pháp được thử chạy.
SIGKILL	9	Nếu một tiến trình nhận signal này, nó phải thoát ra ngay lập tức và sẽ không thực hiện các hoạt động làm sạch.
SIGALRM	14	Tín hiệu báo số lần thực hiện (Alarm Clock).
SIGTERM	15	Tín hiệu kết thúc phần mềm (được gửi bởi sigkill theo mặc định).

Những function an toàn khi gọi trong signal handler

lệnh **kill -l** và nó sẽ hiển thị tất cả các **signal** được hỗ trợ

aio_suspend	fcntl	mkdirat	setgid	tcflow
alarm	fdatasync	mkfifo	setpgid	tcflush
bind	fexecve	mkfifoat	setsid	tcgetattr
cfgetispeed	fork	mknod	setsockopt	tcgetpgrp
cfgetospeed	fstat	mknodat	setuid	tcsendbreak
cfsetispeed	fstatat	open	shutdown	tcsetattr
cfsetospeed	fsync	openat	sigaction	tcsetpgrp
chdir	ftruncate	pause	sigaddset	time
chmod	futimens	pipe	sigdelset	timer_getoverrun
chown	getegid	poll	sigemptyset	timer_gettime
clock_gettime	geteuid	posix_trace_event	sigfillset	timer_settime
close	getgid	pselect	sigismember	times
connect	getgroups	raise	signal	umask
creat	getpeername	read	sigpause	uname
dup	getpgrp	readlink	sigpending	unlink
dup2	getpid	readlinkat	sigprocmask	unlinkat
execl	getppid	recv	sigqueue	utime
execle	getsockname	recvfrom	sigset	utimensat
execv	getsockopt	recvmsg	sigsuspend	utimes
execve	getuid	rename	sleep	wait

12

Ignore a signal

- Sau khi ignore 1 signal number. Mỗi khi signal đó gửi đến, OS sẽ throw nó đi và ko gọi ra signal handler
- Cách ignore 1 signal
 - `signal(singal_number, SIG_IGN);`

Block a signal

- Đôi khi trong 1 thời điểm, process không muốn xử lý 1 số signal nào đó, nó có thể block signal đó.
 - Signal vẫn được gửi đến cho process nhưng sẽ nằm trong hàng chờ, ko được xử lý ngay.
- OS hỗ trợ các function cho phép process block/unblock các signal được gửi đến cho nó.

14

Signal sets

```
#ifndef __ASSEMBLY__  
typedef struct {  
    unsigned long sig[_NSIG_WORDS];  
} sigset_t;
```

- Sigset_t đại diện cho các signal mà process muốn thay đổi trạng thái từ block -> unblock và ngược lại.
 - int sigfillset(sigset_t *set)
 - int sigemptyset(sigset_t *set);
 - int sigaddset(sigset_t *set, int signo)
 - int sigdelset(sigset_t *set, int signo)

Request to block, unblock a signal

- `sigprocmask()` function sẽ request OS update trạng thái block/unblock của các signal nằm trong con trỏ sigmask set vào signal table của process.
 - Các trạng thái block/unblock của sigmask cũ sẽ được lưu vào con trỏ oldset.
 - `int sigprocmask(int how, const sigset_t *set, sigset_t *oldset)`
 - ✓ `SIG_BLOCK`, `SIG_UNBLOCK`, `SIG_SETMASK`
- Thực hành:
 - Viết 1 chương trình C để block Ctrl + C signal từ user
 - Viết 1 chương trình C để in ra các signal mask của process hiện tại

16

Kiểm tra các signal đang bị pending

- `int sigpending(sigset_t *set)`
 - Các signal đang bị pending trong signal table của process sẽ được lưu trong sigmask của con trỏ set
- Một signal đang bị block thì kernel vẫn gửi nó đến cho process nhưng signal đó sẽ bị pending trong hàng đợi mà không được xử lý.
- Thực hành:
 - Viết 1 chương trình C block SIGINT, sau đó check trong ¹⁷số signal pending có SIGINT không, nếu có thì thoát chương trình

Thực hành

- Viết chương trình A fork ra chương trình B. A và B cùng ghi vào 1 file. Mỗi chương trình sẽ đọc lần lượt dòng counter cuối cùng của file và tăng nó lên 1, sau đó ghi tiếp 1 dòng mới là process ID của chính nó kèm biến counter mới. Phải sử dụng signal để đồng bộ giữa 2 process

HỎI - ĐÁP
